ABSTRACT

The face recognition system using ESP32-CAM, OpenCV, and Python is a project aimed at developing an efficient and portable solution for face recognition. The system utilizes the ESP32-CAM module, which combines an ESP32 microcontroller, camera module, and Wi-Fi connectivity, along with the powerful OpenCV library and the Python programming language.

The methodology involves capturing and preprocessing facial images, detecting faces using OpenCV's face detection algorithms, training a face recognition model using machine learning techniques, and implementing real-time face recognition on the ESP32-CAM module.

The system holds potential applications in security, surveillance, access control, and personalized user experiences. The developed system provides a foundation for further research and improvements, such as performance optimization, multi-factor authentication, real-time tracking, and integration with IoT and cloud services.

Through this project, the face recognition system demonstrates its capability to recognize the person and implement it as a attendance system (i.e. it marks the person is present and also note the time when its attendance is recorded).

1.1Background of the Project

The face recognition system using ESP32-CAM, OpenCV, and Python is a project aimed at developing a robust and efficient face recognition solution. Face recognition technology has gained significant attention in recent years due to its wide range of applications in security, surveillance, access control, and personalized user experiences. By combining the capabilities of the ESP32-CAM module, OpenCV library, and Python programming language, we can create a versatile and portable system for facial recognition. [3] The ESP32-CAM module serves as the hardware platform for this project. It integrates an ESP32 microcontroller, camera module, and Wi-Fi connectivity, making it an ideal choice for implementing a face recognition system. The OpenCV library, a popular computer vision library, provides a comprehensive set of functions and algorithms for image processing, including face detection and recognition. Python, a widely used programming language, offers a flexible and user-friendly environment for developing the face recognition system.

The face recognition system involves two main stages: face detection and face recognition. In the face detection stage, the system identifies and locates faces within an image or video stream using computer vision algorithms provided by OpenCV. Once the faces are detected, the face recognition stage begins, where the system matches the detected faces against a pre-existing database of known individuals to identify them. The developed face recognition system has practical implications in various domains. It can enhance security systems by accurately identifying individuals and enabling access control. It can also be applied in surveillance scenarios, enabling real-time monitoring and identification of people. Additionally, the system can provide personalized user experiences by recognizing individuals and customizing interactions accordingly

This project focuses on designing and implementing a face recognition system using ESP32-CAM that can identify individuals and perform actions such as login and logout recording. The system aims to provide a low-cost, efficient, and real-time solution for security and attendance management.

1.2 Problem Statement

Traditional security and attendance systems are not fully reliable due to issues like duplication, manual errors, and lack of automation. There is a need for a smart system that can automatically identify individuals accurately and maintain records without human intervention.

1.3 Objective of the Project

- To develop a real-time face recognition system using ESP32-CAM
- To detect and recognize human faces accurately
- To implement login and logout functionality
- To enhance security using biometric authentication
- To create a low-cost and efficient solution

1.4 Scope of the Project

- This system can be used in:
- Smart home security systems
- Office and college attendance systems
- Access control systems
- Surveillance applications

1.5 Block Diagram

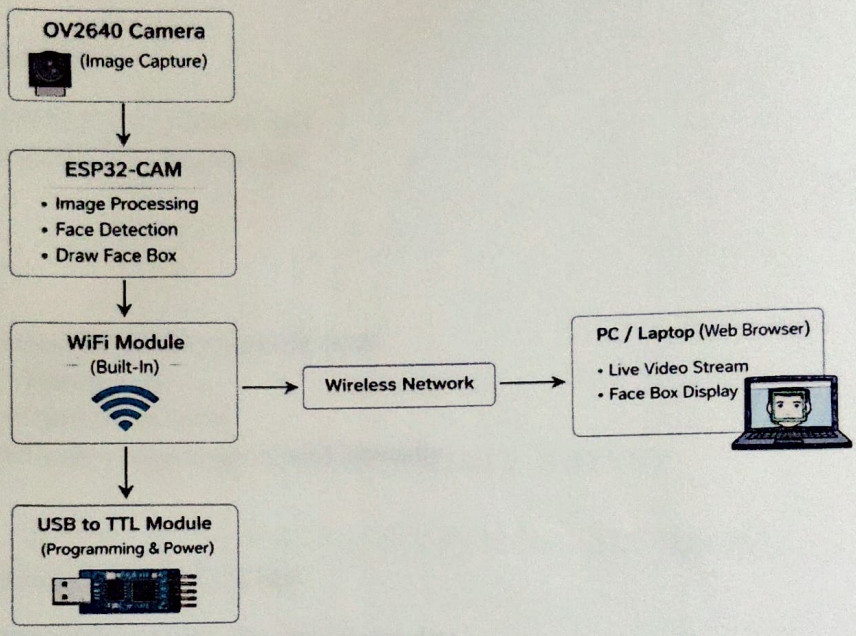


figure 1.5.1 .: Block Diagram of face recognition using ESP-32 cam

The ESP32-CAM module is a compact development board that integrates a microcontroller, camera, and Wi-Fi capabilities. The block diagram mainly shows the power pins, GPIO pins, communication pins, and camera interface.

1. Power Supply Section

- 5V Pin: Used to power the ESP32-CAM module
- 3.3V Pin: Internal regulated voltage output
- GND (Ground): Common ground connection

2. GPIO (General Purpose Input Output) Pins

- Pins like GPIO 12, 13, 14, 15, 2, 4, 16, 0, 1, 3 are used
- These pins can act as input or output
- Used for connecting sensors, LEDs, relays, etc.

3. Camera Interface (CSI - Camera Serial Interface)

- Pins labeled HS2_DATA0 to HS2_DATA3, HS2_CLK, HS2_CMD
- These are used to transfer image data from the camera to ESP32

4. UART Communication

- U0TXD (GPIO 1) → Transmit data
- U0RXD (GPIO 3) → Receive data

5. Control Pins

- GPIO 0:Important for programming mode
- LOW → Upload code
- HIGH → Normal execution
- GPIO 16:General purpose pin or used internally

6. Wi-Fi & Processing Unit (ESP32 Chip)

- The ESP32 microcontroller:Processes image data
- Runs face detection/recognition algorithms
- Sends data over Wi-Fi

3.1 Scope of the project

The face recognition system using ESP32-CAM, OpenCV, and Python has promising potential for further development and enhancement. Here are some future scopes to consider: Real-Time Tracking: Extend the system to incorporate real-time face tracking capabilities. This would enable the system to track and recognize faces in a video stream or in dynamic environments, such as surveillance applications.

Emotion Recognition: Integrate emotion recognition capabilities into the system to detect and analyze human emotions from facial expressions. This can have applications in fields such as market research, healthcare, and human-computer interaction. Privacy Enhancements: Address privacy concerns by implementing privacy preserving techniques, such as face anonymization or secure storage and transmission of facial data.

Integration with IoT and Cloud Services: This can enable remote monitoring and control, centralized management, data analysis, and integration with other smart systems. Cloud-based solutions can provide scalability and flexibility for large-scale face recognition deployments. Application-Specific Customization: Customize the system for specific applications, such as attendance management systems, access control systems, or personalized user experiences in smart devices. This would involve tailoring the system's functionalities and user interfaces to meet the specific requirements of different use cases.

By exploring these future scopes, the face recognition system using ESP32- CAM, OpenCV, and Python can be further improved, expanded, and applied to various domains, contributing to advancements in security, automation, and user-centric technologies.

4.4 Flow chart

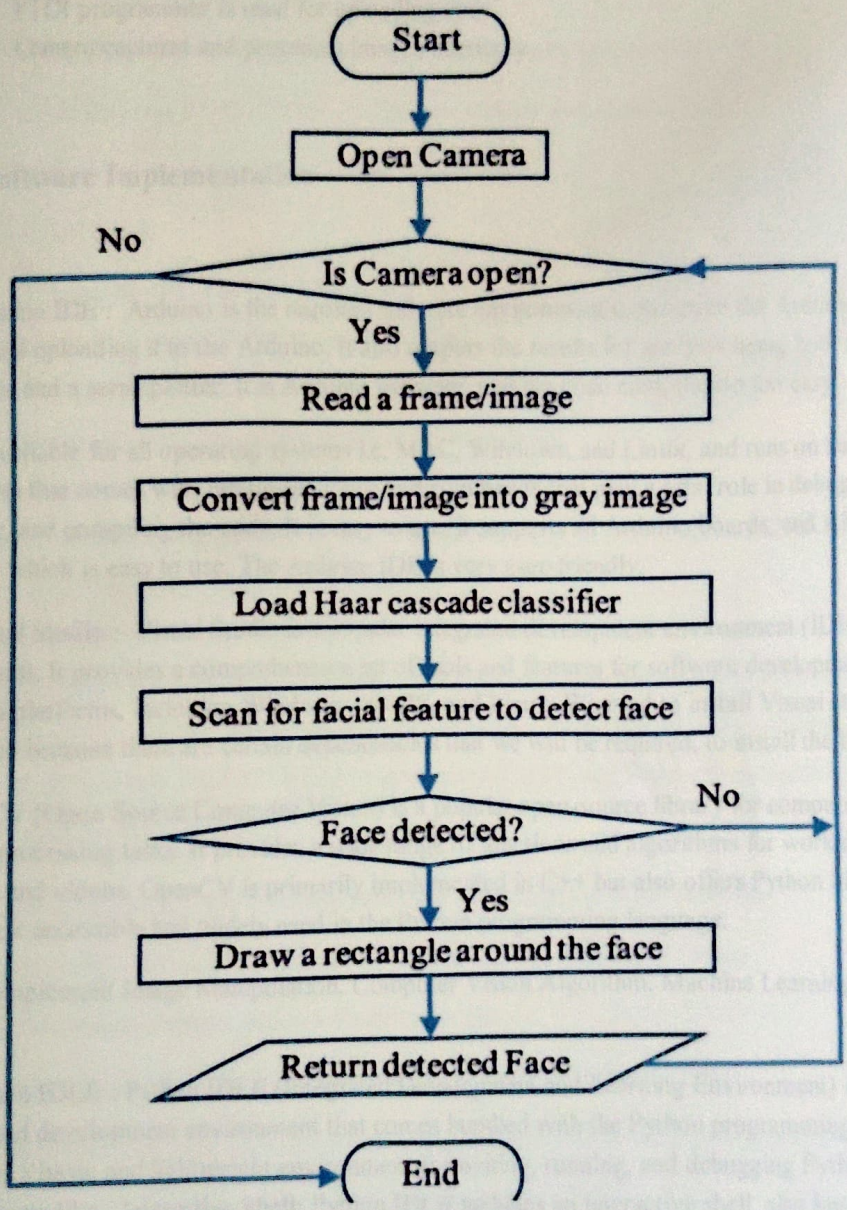


fig.2.4.4 Flowchart of Operational Process

5.1 Details of designs, working and processes

In this Face Recognition Based Attendance System using ESP32 CAM Module. We will be using OpenCV & Visual Studio for this application. OpenCV is an open-sourced image processing library that is very widely used not just in industry but also in the field of research and development. Visual Studio is an IDE made by Microsoft for different types of software development & contains completion tools, compilers, and other features to facilitate the software development process. Face Recognition Based Attendance system using ESP32 CAM and Python. The main heavy program will be at the server-side that is our computer, or one can even use raspberry-pi as a server. In this attendance system, we will not just detect the person but also store the information of the person detected in a Microsoft Excel File. Moreover, the duration of time they have stayed in the frame is also recorded into an excel sheet. Here we explain step by step how we run the program and take the attendance in the excel sheet.

Step 1: Installing ESP32CAM Library

Here we will not use the general ESP webserver example rather another streaming process. Therefore we need to add another ESPCAM library. The esp32cam library provides an object oriented API to use OV2640 camera on ESP32 microcontroller. It is a wrapper of esp32- camera library. We download this library from Github

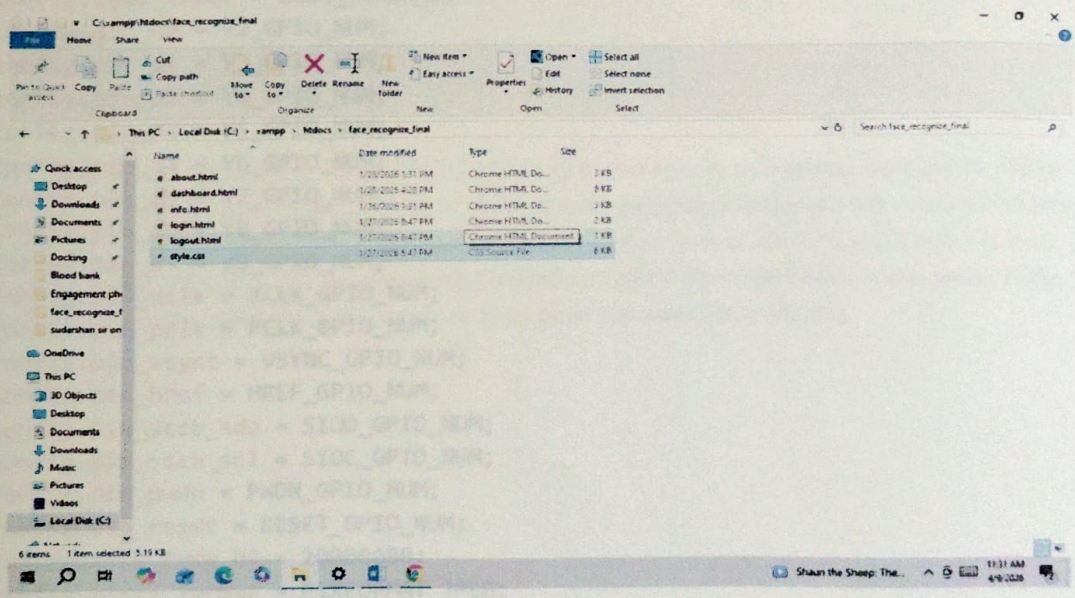


fig 5 .1: Installing ESP32CAM Library

Once downloaded add this zip library to Arduino Library Folder. To do so follow the following steps: Open Arduino -> Sketch -> Include Library -> Add .ZIP Library... -> Navigate to downloaded zip file -> add

Step 2 :Source Code/Program for ESP32 CAM Module

Here is a source code for Face Recognition Based Attendance System using ESP32 CAM & OpenCV

```
#include "esp_camera.h"
#include <WiFi.h>

// Camera Model - AI Thinker
#define CAMERA_MODEL_AI_THINKER
#include "camera_pins.h"

// WiFi Credentials
const char* ssid = "poco";
const char* password = "12341234";

void startCameraServer();

void setup() {
  Serial.begin(115200);

  camera_config_t config;
  config.ledc_channel = LEDC_CHANNEL_0;
  config.ledc_timer = LEDC_TIMER_0;
  config.pin_d0 = Y2_GPIO_NUM;
  config.pin_d1 = Y3_GPIO_NUM;
  config.pin_d2 = Y4_GPIO_NUM;
  config.pin_d3 = Y5_GPIO_NUM;
  config.pin_d4 = Y6_GPIO_NUM;
  config.pin_d5 = Y7_GPIO_NUM;
  config.pin_d6 = Y8_GPIO_NUM;
  config.pin_d7 = Y9_GPIO_NUM;
  config.pin_xclk = XCLK_GPIO_NUM;
  config.pin_pclk = PCLK_GPIO_NUM;
  config.pin_vsync = VSYNC_GPIO_NUM;
  config.pin_href = HREF_GPIO_NUM;
  config.pin_sccb_sda = SIOD_GPIO_NUM;
  config.pin_sccb_scl = SIOC_GPIO_NUM;
  config.pin_pwdn = PWDN_GPIO_NUM;
  config.pin_reset = RESET_GPIO_NUM;
  config.xclk_freq_hz = 20000000;
  config.pixel_format = PIXFORMAT_JPEG;

  // Lag कमी करण्यासाठी रिझोल्यूशन बदलले
  if(psramFound()){
    config.frame_size = FRAMESIZE_QVGA; // 320x240 (Fastest for Web)
    config.jpeg_quality = 15;
    config.fb_count = 2;
```

```
  } else {
    config.frame_size = FRAMESIZE_QVGA;
    config.jpeg_quality = 12;
    config.fb_count = 1;
  }

  esp_err_t err = esp_camera_init(&config);
  if (err != ESP_OK) {
    Serial.printf("Camera init failed!");
    return;
  }

  // WiFi Connect
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }

  startCameraServer();
  Serial.println("\nWiFi Connected!");
  Serial.print("Use this IP: ");
  Serial.println(WiFi.localIP());
}

void loop() {
  delay(10000);
}
```

Now compile and upload it to the ESP32 CAM Board. But during uploading, we have to follow few steps every time. Make sure the IO0 pin is shorted with the ground when pressed the upload button. If it give the dots and dashes while uploading press the reset button immediately.Once the code is uploaded, remove the I01 pin shorting with Ground and press the reset button once again. If the output is the Serial monitor is still not there then press the reset button again.

Step 3 : Installing Visual Studio

Now in order to proceed further, we need to install Visual studio. We are doing so because there are certain dependencies that we will be required, to install the libraries, later in this tutorial. To install go to this Visual Studio website. Then, download the latest community version.

Once downloaded install the software a welcome installation screen would appear, now select the community version. After this from numerous selecting boxes, select Desktop development with C++. At the right-hand side select the boxes as in the image below. Fig Serial Monitor Output 20 Now that we have selected the click on Install/Modify at the Right corner below. This process will take time as it will download large files. Once installed it will ask for restarting the PC. So that will be done automatically

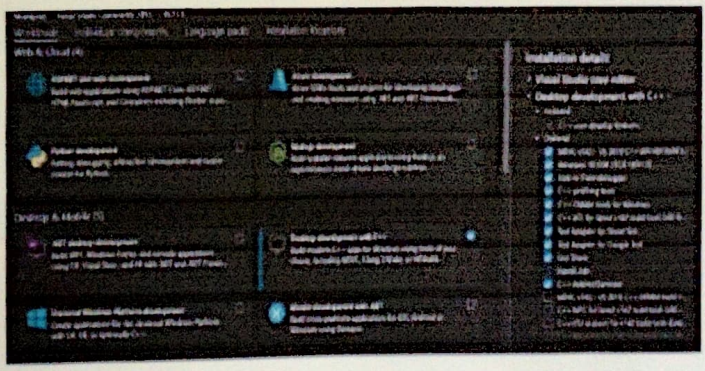


fig 6.5.2: Installing Visual Studio

Step 4: Installing required libraries and run program:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Dashboard - Face Recognition</title>
  <link rel="stylesheet" href="style.css">
  <script defer src="https://cdn.jsdelivr.net/npm/@vladmandic/face-api/dist/face-api.js"></script>
</head>
<body>
  <div class="app-container">
    <!-- SIDEBAR -->
    <nav class="sidebar">
      <h2>FaceRecog</h2>
      <ul class="nav-links">
        <li><a href="dashboard.html" class="active">Dashboard</a></li>
        <li><a href="info.html">What is Face Recog?</a></li>
        <li><a href="about.html">About/Student Info</a></li>
      </ul>
    </nav>
```

```
<!-- MAIN CONTENT -->
    <main class="main-content">
      <h1 class="page-title">ESP32_CAM</h1>
      <div class="camera-wrapper">
        <!-- VIDEO AREA -->
        <div class="video-section">
          <div class="video-container" id="container">
            <video id="video" autoplay muted playsinline></video>
          </div>
        </div>

        <!-- CONTROLS -->
        <div class="controls-section">
          <h3>Add New Face</h3>
          <p>Register a new student or user into the system.</p>
          <input type="text" id="name" placeholder="Enter Name">
          <button class="btn-primary" onclick="captureFace()">Capture & Save</button>
          <p class="status-text" id="status">Loading models...</p>
        </div>
      </div>
    </main>
  </div>
```

```
<script>
    const video = document.getElementById("video");
    const statusText = document.getElementById("status");
    const container = document.getElementById("container");

    let canvas, ctx;
    let labeledDescriptors = [];
    let faceMatcher = null;

    // CAMERA
    navigator.mediaDevices.getUserMedia({ video: true })
      .then(stream => { video.srcObject = stream; })
      .catch(err => console.error("Camera error:", err));

  // LOAD MODELS
  async function loadModels() {
    const URL = "https://justadudewhohacks.github.io/face-api.js/weights/"; // more stable than raw github

    await Promise.all([
      faceapi.nets.ssdMobilenetv1.loadFromUri(URL),
      faceapi.nets.faceLandmark68Net.loadFromUri(URL),
      faceapi.nets.faceRecognitionNet.loadFromUri(URL)
    ]);
```

```
    statusText.innerText = "Models Loaded";
    loadSavedFaces();   // now safe to call
}

function loadSavedFaces() {
    const saved = localStorage.getItem("faces");
    if (!saved) {
        statusText.innerText = "No saved faces yet";
        return;
    }

    try {
        const parsed = JSON.parse(saved);

        labeledDescriptors = parsed.map(item => {
            // === THE IMPORTANT FIX ===
            // Convert plain arrays back → Float32Array
            const descriptors = item.descriptors.map(arr => new Float32Array(arr));
            return new faceapi.LabeledFaceDescriptors(item.label, descriptors);
        });

        faceMatcher = new faceapi.FaceMatcher(labeledDescriptors, 0.6);

        statusText.innerText = `Loaded ${labeledDescriptors.length} saved face(s)`;
        console.log("Loaded:", labeledDescriptors.map(ld => ld.label));
    } catch (e) {
        console.error("Cannot load saved faces:", e);
        statusText.innerText = "Error loading saved faces (corrupted data?)";
```

```
        // Optional: localStorage.removeItem("faces"); // clean if broken
      }
    }

    function saveFaces() {
      const dataToSave = labeledDescriptors.map(ld => ({
        label: ld.label,
        descriptors: ld.descriptors.map(d => Array.from(d)) // Float32Array → normal Array for
JSON
      }));

      localStorage.setItem("faces", JSON.stringify(dataToSave));
      console.log("Faces saved to localStorage");
    }

    // START AFTER VIDEO READY
    video.addEventListener("loadedmetadata", async () => {
      canvas = faceapi.createCanvasFromMedia(video);
      container.appendChild(canvas);
      canvas.width = video.videoWidth;
      canvas.height = video.videoHeight;
      ctx = canvas.getContext("2d");

      await loadModels();
      setInterval(detectFaces, 80);
    });
```

```
// FACE DETECTION LOOP  - unchanged
    async function detectFaces() {
      const detections = await faceapi
        .detectAllFaces(video, new faceapi.SsdMobilenetv1Options({ minConfidence: 0.6 }))
        .withFaceLandmarks()
        .withFaceDescriptors();

      ctx.clearRect(0, 0, canvas.width, canvas.height);

      detections.forEach(det => {
        let label = "Unknown";
        let color = "#ff3333";

        if (faceMatcher) {
          const result = faceMatcher.findBestMatch(det.descriptor);
          if (result.label !== "unknown") {
            label = result.label;
            color = "#00ff88";
          }
        }
```

```
// FACE DETECTION LOOP - unchanged
    async function detectFaces() {
      const detections = await faceapi
        .detectAllFaces(video, new faceapi.SsdMobilenetv1Options({ minConfidence: 0.6 }))
        .withFaceLandmarks()
        .withFaceDescriptors();

      ctx.clearRect(0, 0, canvas.width, canvas.height);

      detections.forEach(det => {
        let label = "Unknown";
        let color = "#ff3333";

        if (faceMatcher) {
          const result = faceMatcher.findBestMatch(det.descriptor);
          if (result.label !== "unknown") {
            label = result.label;
            color = "#00ff88";
          }
        }
```

```
      const box = d...detection.box;
      const padX = box.width * 0.3;
      const padY = box.height * 0.35;

      let x = box.x - padX;
      let y = box.y - padY;
      let w = box.width + padX * 2;
      let h = box.height + padY * 2;

      // MIRROR FIX
      let drawX = canvas.width - (x + w);

      ctx.strokeStyle = color;
      ctx.lineWidth = 3;
      ctx.strokeRect(drawX, y, w, h);

      ctx.fillStyle = color;
      ctx.fillRect(drawX, y - 28, ctx.measureText(label).width + 20, 26);

      ctx.fillStyle = "#000";
      ctx.font = "bold 16px Arial";
      ctx.fillText(label, drawX + 10, y - 10);
    });
  }
```

```
// CAPTURE FACE – unchanged
async function captureFace() {
  const name = document.getElementById("name").value.trim();
  if (!name) {
    alert("Enter name");
    return;
  }

  const tempCanvas = document.createElement("canvas");
  tempCanvas.width = video.videoWidth;
  tempCanvas.height = video.videoHeight;
  const tctx = tempCanvas.getContext("2d");

  // mirror save
  tctx.translate(tempCanvas.width, 0);
  tctx.scale(-1, 1);
  tctx.drawImage(video, 0, 0);

  const detection = await faceapoidetectSingleFace(tempCanvas, new
faceapi.SsdMobilenetv1Options({ minConfidence: 0.6 }))
    .withFaceLandmarks()
    .withFaceDescriptor();
```

```
  if (!detection) {
        statusText.innerText = "No face detected";
        return;
      }

      labeledDescriptors.push(
        new faceapi.LabeledFaceDescriptors(name, [detection.descriptor])
      );



    faceMatcher = new faceapi.FaceMatcher(labeledDescriptors, 0.6);

      saveFaces();

      statusText.innerText = "Saved & Ready: " + name;
      document.getElementById("name").value = "";
    }
  </script>
</body>
</html>
```

5.2 Working Model of face recognition using ESP-32 cam

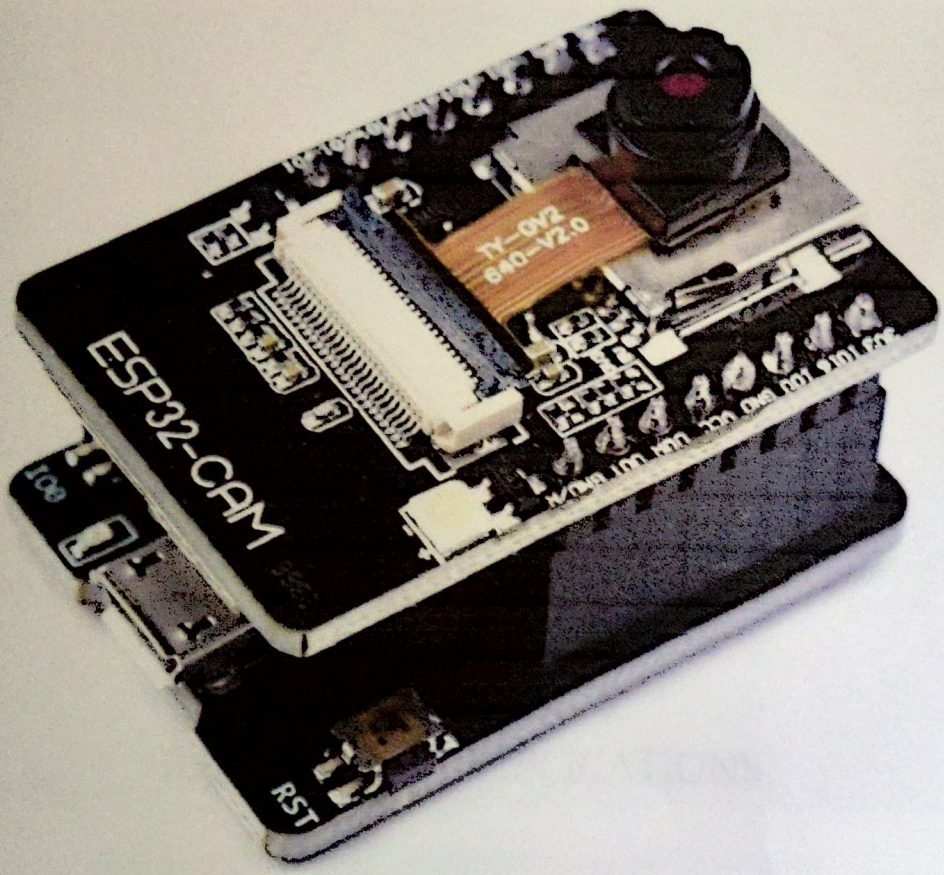


figure 7 .5.3 :Working Model of face recognition using ESP-32 cam

QR CODE

